Open in Colab

T81-558: Applications of Deep Neural Networks

Module 13: Advanced/Other Topics

- Instructor: Jeff Heaton, McKelvey School of Engineering, Washington University in St. Louis
- For more information visit the class website.

Module 13 Video Material

- Part 13.1: Flask and Deep Learning Web Services [Video] [Notebook]
- Part 13.2: Interrupting and Continuing Training [Video] [Notebook]
- Part 13.3: Using a Keras Deep Neural Network with a Web Application [Video] [Notebook]
- Part 13.4: When to Retrain Your Neural Network [Video] [Notebook]
- **Part 13.5: Tensor Processing Units (TPUs)** [Video] [Notebook]

Google CoLab Instructions

The following code ensures that Google CoLab is running the correct version of TensorFlow.

In [ ]:
```
# Detect Colab if present
try:
    from google.colab import drive
    COLAB = True
    print("Note: using Google CoLab")
    %tensorflow_version 2.x
except:
    print("Note: not using Google CoLab")
    COLAB = False
```

```
Note: using Google CoLab
```

To use Tensor Processing Units (TPUs), you must grant access to Google Cloud Platform (GCP) drives. If this access is not successfully, you will likely see this error:

```
InvalidArgumentError: Unable to parse tensor proto
```

From Google CoLab, issue the following command:

In [ ]:
```
from google.colab import auth

auth.authenticate_user()
```

Part 13.5: Tensor Processing Units (TPUs)

This book focuses primarily on NVIDIA Graphics Processing Units (GPUs) for deep learning acceleration. NVIDIA GPUs are not the only option for deep learning acceleration. TensorFlow continues to gain additional support for AMD and Intel GPUs. TPUs are also available from Google cloud platforms to accelerate deep learning. The focus of this book and course is on NVIDIA GPUs because of their wide availability on both local and cloud systems.

Though this book focuses on NVIDIA GPUs, we will briefly examine Google Tensor Processing Units (TPUs). These devices are an AI accelerator Application-Specific Integrated Circuit (ASIC) developed by Google. They were designed specifically for neural network machine learning, mainly using Google's TensorFlow software. Google began using TPUs internally in 2015 and in 2018 made them available for third-party use, both as part of its cloud infrastructure and by offering a smaller version of the chip for sale.

The full use of a TPU is a complex topic that I only introduced in this part. Supporting TPUs is slightly more complicated than GPUs because specialized coding is needed. Changes are rarely required to adapt CPU code to GPU for most relatively simple mainstream GPU tasks in TensorFlow. I will cover the mild code changes needed to utilize in this part.

We will create a regression neural network to count paper clips in this part. I demonstrated this dataset and task several times previously in this book. This part focuses on the utilization of TPUs and not the creation of neural networks. I covered the design of computer vision previously in this book.

In [ ]:
```
# HIDE OUTPUT
import os
import pandas as pd

URL = "https://github.com/jeffheaton/data-mirror/"
DOWNLOAD_SOURCE = URL+"releases/download/v1/paperclips.zip"
DOWNLOAD_NAME = DOWNLOAD_SOURCE[DOWNLOAD_SOURCE.rfind('/')+1:]

if COLAB:
  PATH = "/content"
else:
  # I used this locally on my machine, you may need different
```

```
  PATH = "/Users/jeff/temp"

EXTRACT_TARGET = os.path.join(PATH,"clips")
SOURCE = os.path.join(EXTRACT_TARGET, "paperclips")

# Download paperclip data
!wget -O {os.path.join(PATH,DOWNLOAD_NAME)} {DOWNLOAD_SOURCE}
!mkdir -p {SOURCE}
!mkdir -p {TARGET}
!mkdir -p {EXTRACT_TARGET}
!unzip -o -j -d {SOURCE} {os.path.join(PATH, DOWNLOAD_NAME)} >/dev/null

# Add filenames
df_train = pd.read_csv(os.path.join(SOURCE, "train.csv"))
df_train['filename'] = "clips-" + df_train.id.astype(str) + ".jpg"
```

```
--2022-03-13 17:29:19--  https://github.com/jeffheaton/data-mirror/releases/downloa
d/v1/paperclips.zip
Resolving github.com (github.com)... 140.82.112.3
Connecting to github.com (github.com)|140.82.112.3|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://objects.githubusercontent.com/github-production-release-asset-2e65
be/408419764/25830812-b9e6-4ddf-93b6-7932d9ef5982?X-Amz-Algorithm=AWS4-HMAC-SHA256&X
-Amz-Credential=AKIAIWNJYAX4CSVEH53A%2F20220313%2Fus-east-1%2Fs3%2Faws4_request&X-Am
z-Date=20220313T172920Z&X-Amz-Expires=300&X-Amz-Signature=c08a2d9adeff4affd60042247c
d7b0f98d9acb45c89cbf137eadd5d3fe1ccac0&X-Amz-SignedHeaders=host&actor_id=0&key_id=0&
repo_id=408419764&response-content-disposition=attachment%3B%20filename%3Dpaperclip
s.zip&response-content-type=application%2Foctet-stream [following]
--2022-03-13 17:29:20--  https://objects.githubusercontent.com/github-production-rel
ease-asset-2e65be/408419764/25830812-b9e6-4ddf-93b6-7932d9ef5982?X-Amz-Algorithm=AWS
4-HMAC-SHA256&X-Amz-Credential=AKIAIWNJYAX4CSVEH53A%2F20220313%2Fus-east-1%2Fs3%2Faw
s4_request&X-Amz-Date=20220313T172920Z&X-Amz-Expires=300&X-Amz-Signature=c08a2d9adef
f4affd60042247cd7b0f98d9acb45c89cbf137eadd5d3fe1ccac0&X-Amz-SignedHeaders=host&actor
_id=0&key_id=0&repo_id=408419764&response-content-disposition=attachment%3B%20filena
me%3Dpaperclips.zip&response-content-type=application%2Foctet-stream
Resolving objects.githubusercontent.com (objects.githubusercontent.com)... 185.199.1
08.133, 185.199.109.133, 185.199.110.133, ...
Connecting to objects.githubusercontent.com (objects.githubusercontent.com)|185.199.
108.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 163590691 (156M) [application/octet-stream]
Saving to: ‘/content/paperclips.zip’

/content/paperclips 100%[===================>] 156.01M   211MB/s    in 0.7s

2022-03-13 17:29:21 (211 MB/s) - ‘/content/paperclips.zip’ saved [163590691/16359069
1]
```

Preparing Data for TPUs

To present the paperclips dataset to the TPU, we will convert the images to a Keras Dataset. Because we will load the entire dataset to RAM, we will only utilize the first

1,000 images. We previously loaded the labels from the **train.csv** file. The following code loads these images and converts them to a Keras dataset.

In [ ]:
```
import tensorflow as tf
import keras_preprocessing
import glob, os
import tqdm
import numpy as np
from PIL import Image

IMG_SHAPE = (128,128)
BATCH_SIZE = 32

# Resize each image and convert the 0-255 ranged RGB values to 0-1 range.
def load_images(files, img_shape):
  cnt = len(files)
  x = np.zeros((cnt,)+img_shape+(3,),dtype=np.float32)
  i = 0
  for file in tqdm.tqdm(files):
    img = Image.open(file)
    img = img.resize(img_shape)
    img = np.array(img)
    img = img/255
    x[i,:,:,:] = img
    i+=1
  return x

# Process training data
df_train = pd.read_csv(os.path.join(SOURCE, "train.csv"))
df_train['filename'] = "clips-" + df_train.id.astype(str) + ".jpg"

# Use only the first 1000 images
df_train = df_train[0:1000]

# Load images
images = [os.path.join(SOURCE,x) for x in df_train.filename]
x = load_images(images, IMG_SHAPE)
y = df_train.clip_count.values

# Convert to dataset
dataset = tf.data.Dataset.from_tensor_slices((x, y))
dataset = dataset.batch(BATCH_SIZE)
```

```
100%|██████████| 1000/1000 [00:02<00:00, 413.74it/s]
```

TPUs are typically Cloud TPU workers, different from the local process running the user's Python program. Thus, it would be best to do some initialization work to connect to the remote cluster and initialize the TPUs. The TPU argument to **tf.distribute.cluster_resolver**. **TPUClusterResolver** is a unique address just for Colab. If you are running your code on Google Compute Engine (GCE), you should instead pass in the name of your Cloud TPU. The following code performs this initialization.

In [ ]:
```
# HIDE OUTPUT
try:
    tpu = tf.distribute.cluster_resolver.TPUClusterResolver.connect()
    print("Device:", tpu.master())
    strategy = tf.distribute.TPUStrategy(tpu)
except:
    strategy = tf.distribute.get_strategy()
print("Number of replicas:", strategy.num_replicas_in_sync)
```

```
INFO:tensorflow:Deallocate tpu buffers before initializing tpu system.
INFO:tensorflow:Deallocate tpu buffers before initializing tpu system.
INFO:tensorflow:Initializing the TPU system: grpc://10.97.246.186:8470
INFO:tensorflow:Initializing the TPU system: grpc://10.97.246.186:8470
INFO:tensorflow:Finished initializing TPU system.
INFO:tensorflow:Finished initializing TPU system.
Device: grpc://10.97.246.186:8470
INFO:tensorflow:Found TPU system:
INFO:tensorflow:Found TPU system:
INFO:tensorflow:*** Num TPU Cores: 8
INFO:tensorflow:*** Num TPU Cores: 8
INFO:tensorflow:*** Num TPU Workers: 1
INFO:tensorflow:*** Num TPU Workers: 1
INFO:tensorflow:*** Num TPU Cores Per Worker: 8
INFO:tensorflow:*** Num TPU Cores Per Worker: 8
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:localhost/replica:0/tas
k:0/device:CPU:0, CPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:localhost/replica:0/tas
k:0/device:CPU:0, CPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:CPU:0, CPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:CPU:0, CPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:0, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:0, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:1, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:1, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:2, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:2, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:3, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:3, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:4, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:4, TPU, 0, 0)
```

```
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:5, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:5, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:6, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:6, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:7, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU:7, TPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU_SYSTEM:0, TPU_SYSTEM, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:TPU_SYSTEM:0, TPU_SYSTEM, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:XLA_CPU:0, XLA_CPU, 0, 0)
INFO:tensorflow:*** Available Device: _DeviceAttributes(/job:worker/replica:0/task:
0/device:XLA_CPU:0, XLA_CPU, 0, 0)
Number of replicas: 8
```

We will now use a ResNet neural network as a basis for our neural network. We begin by loading, from Keras, the ResNet50 network. We will redefine both the input shape and output of the ResNet model, so we will not transfer the weights. Since we redefine the input, the weights are of minimal value. We specify **include_top** as False because we will change the input resolution. We also specify **weights** as false because we must retrain the network after changing the top input layers.

In [ ]:
```
from tensorflow.keras.applications.resnet50 import ResNet50
from tensorflow.keras.layers import Input
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.metrics import RootMeanSquaredError

def create_model():
  input_tensor = Input(shape=IMG_SHAPE+(3,))

  base_model = ResNet50(
      include_top=False, weights=None, input_tensor=input_tensor,
      input_shape=None)

  x=base_model.output
  x=GlobalAveragePooling2D()(x)
  x=Dense(1024,activation='relu')(x)
  x=Dense(1024,activation='relu')(x)
  model=Model(inputs=base_model.input,outputs=Dense(1)(x))
  return model

with strategy.scope():
  model = create_model()
```

```
model.compile(loss = 'mean_squared_error', optimizer='adam',
            metrics=[RootMeanSquaredError(name="rmse")])

history = model.fit(dataset, epochs=100, steps_per_epoch=32,
                  verbose = 1)
```

```
Epoch 1/100
32/32 [==============================] - 91s 1s/step - loss: 412.2361 - rmse: 20.303
6
Epoch 2/100
32/32 [==============================] - 1s 43ms/step - loss: 156.3177 - rmse: 12.50
27
Epoch 3/100
32/32 [==============================] - 1s 43ms/step - loss: 126.0154 - rmse: 11.22
57
Epoch 4/100
32/32 [==============================] - 1s 44ms/step - loss: 81.0153 - rmse: 9.0008
Epoch 5/100
32/32 [==============================] - 1s 44ms/step - loss: 56.0450 - rmse: 7.4863
Epoch 6/100
32/32 [==============================] - 1s 44ms/step - loss: 54.7298 - rmse: 7.3980
Epoch 7/100
32/32 [==============================] - 1s 44ms/step - loss: 51.7157 - rmse: 7.1914
Epoch 8/100
32/32 [==============================] - 1s 44ms/step - loss: 77.8686 - rmse: 8.8243
Epoch 9/100
32/32 [==============================] - 1s 44ms/step - loss: 49.4168 - rmse: 7.0297
Epoch 10/100
32/32 [==============================] - 1s 44ms/step - loss: 54.6380 - rmse: 7.3918
Epoch 11/100
32/32 [==============================] - 1s 45ms/step - loss: 53.8622 - rmse: 7.3391
Epoch 12/100
32/32 [==============================] - 1s 44ms/step - loss: 42.4151 - rmse: 6.5127
Epoch 13/100
32/32 [==============================] - 1s 45ms/step - loss: 67.3325 - rmse: 8.2056
Epoch 14/100
32/32 [==============================] - 2s 46ms/step - loss: 113.7659 - rmse: 10.66
61
Epoch 15/100
32/32 [==============================] - 1s 45ms/step - loss: 78.4862 - rmse: 8.8592
Epoch 16/100
32/32 [==============================] - 1s 44ms/step - loss: 76.3148 - rmse: 8.7358
Epoch 17/100
32/32 [==============================] - 1s 44ms/step - loss: 53.6310 - rmse: 7.3233
Epoch 18/100
32/32 [==============================] - 1s 44ms/step - loss: 35.9240 - rmse: 5.9937
Epoch 19/100
32/32 [==============================] - 1s 44ms/step - loss: 28.7515 - rmse: 5.3620
Epoch 20/100
32/32 [==============================] - 1s 44ms/step - loss: 24.2422 - rmse: 4.9236
Epoch 21/100
32/32 [==============================] - 1s 45ms/step - loss: 34.2141 - rmse: 5.8493
Epoch 22/100
32/32 [==============================] - 1s 44ms/step - loss: 25.2991 - rmse: 5.0298
Epoch 23/100
32/32 [==============================] - 1s 44ms/step - loss: 15.2113 - rmse: 3.9002
Epoch 24/100
32/32 [==============================] - 1s 43ms/step - loss: 16.7044 - rmse: 4.0871
Epoch 25/100
32/32 [==============================] - 1s 44ms/step - loss: 17.5892 - rmse: 4.1940
Epoch 26/100
32/32 [==============================] - 1s 44ms/step - loss: 18.7181 - rmse: 4.3264
```

```
Epoch 27/100
32/32 [==============================] - 1s 44ms/step - loss: 16.7338 - rmse: 4.0907
Epoch 28/100
32/32 [==============================] - 1s 45ms/step - loss: 15.6866 - rmse: 3.9606
Epoch 29/100
32/32 [==============================] - 1s 45ms/step - loss: 26.5499 - rmse: 5.1527
Epoch 30/100
32/32 [==============================] - 1s 44ms/step - loss: 22.8419 - rmse: 4.7793
Epoch 31/100
32/32 [==============================] - 1s 45ms/step - loss: 32.6161 - rmse: 5.7111
Epoch 32/100
32/32 [==============================] - 1s 45ms/step - loss: 45.6118 - rmse: 6.7537
Epoch 33/100
32/32 [==============================] - 1s 44ms/step - loss: 13.8262 - rmse: 3.7184
Epoch 34/100
32/32 [==============================] - 1s 45ms/step - loss: 13.2402 - rmse: 3.6387
Epoch 35/100
32/32 [==============================] - 1s 45ms/step - loss: 25.0043 - rmse: 5.0004
Epoch 36/100
32/32 [==============================] - 1s 44ms/step - loss: 16.8132 - rmse: 4.1004
Epoch 37/100
32/32 [==============================] - 1s 44ms/step - loss: 16.7914 - rmse: 4.0977
Epoch 38/100
32/32 [==============================] - 1s 44ms/step - loss: 44.0505 - rmse: 6.6371
Epoch 39/100
32/32 [==============================] - 1s 44ms/step - loss: 37.8859 - rmse: 6.1552
Epoch 40/100
32/32 [==============================] - 1s 43ms/step - loss: 45.2483 - rmse: 6.7267
Epoch 41/100
32/32 [==============================] - 1s 44ms/step - loss: 40.9563 - rmse: 6.3997
Epoch 42/100
32/32 [==============================] - 1s 44ms/step - loss: 29.2618 - rmse: 5.4094
Epoch 43/100
32/32 [==============================] - 1s 44ms/step - loss: 27.8391 - rmse: 5.2763
Epoch 44/100
32/32 [==============================] - 1s 45ms/step - loss: 74.5348 - rmse: 8.6334
Epoch 45/100
32/32 [==============================] - 1s 44ms/step - loss: 67.7960 - rmse: 8.2338
Epoch 46/100
32/32 [==============================] - 1s 45ms/step - loss: 92.6046 - rmse: 9.6231
Epoch 47/100
32/32 [==============================] - 1s 45ms/step - loss: 84.2020 - rmse: 9.1762
Epoch 48/100
32/32 [==============================] - 1s 45ms/step - loss: 85.4522 - rmse: 9.2440
Epoch 49/100
32/32 [==============================] - 1s 44ms/step - loss: 71.1020 - rmse: 8.4322
Epoch 50/100
32/32 [==============================] - 1s 44ms/step - loss: 108.8298 - rmse: 10.43
21
Epoch 51/100
32/32 [==============================] - 1s 44ms/step - loss: 79.8435 - rmse: 8.9355
Epoch 52/100
32/32 [==============================] - 1s 44ms/step - loss: 55.1271 - rmse: 7.4248
Epoch 53/100
32/32 [==============================] - 1s 44ms/step - loss: 45.2738 - rmse: 6.7286
Epoch 54/100
```

```
32/32 [==============================] - 1s 45ms/step - loss: 40.6076 - rmse: 6.3724
Epoch 55/100
32/32 [==============================] - 1s 45ms/step - loss: 49.2865 - rmse: 7.0204
Epoch 56/100
32/32 [==============================] - 1s 45ms/step - loss: 52.1646 - rmse: 7.2225
Epoch 57/100
32/32 [==============================] - 1s 45ms/step - loss: 30.5701 - rmse: 5.5290
Epoch 58/100
32/32 [==============================] - 1s 44ms/step - loss: 31.0364 - rmse: 5.5710
Epoch 59/100
32/32 [==============================] - 1s 45ms/step - loss: 25.4160 - rmse: 5.0414
Epoch 60/100
32/32 [==============================] - 1s 44ms/step - loss: 22.1702 - rmse: 4.7085
Epoch 61/100
32/32 [==============================] - 1s 45ms/step - loss: 24.5868 - rmse: 4.9585
Epoch 62/100
32/32 [==============================] - 1s 45ms/step - loss: 18.4491 - rmse: 4.2952
Epoch 63/100
32/32 [==============================] - 1s 44ms/step - loss: 25.1780 - rmse: 5.0178
Epoch 64/100
32/32 [==============================] - 1s 44ms/step - loss: 14.7577 - rmse: 3.8416
Epoch 65/100
32/32 [==============================] - 1s 45ms/step - loss: 15.3932 - rmse: 3.9234
Epoch 66/100
32/32 [==============================] - 1s 45ms/step - loss: 22.6359 - rmse: 4.7577
Epoch 67/100
32/32 [==============================] - 1s 44ms/step - loss: 13.0876 - rmse: 3.6177
Epoch 68/100
32/32 [==============================] - 1s 44ms/step - loss: 19.2264 - rmse: 4.3848
Epoch 69/100
32/32 [==============================] - 1s 43ms/step - loss: 15.3374 - rmse: 3.9163
Epoch 70/100
32/32 [==============================] - 1s 44ms/step - loss: 14.3568 - rmse: 3.7890
Epoch 71/100
32/32 [==============================] - 1s 44ms/step - loss: 19.8507 - rmse: 4.4554
Epoch 72/100
32/32 [==============================] - 1s 45ms/step - loss: 13.0564 - rmse: 3.6134
Epoch 73/100
32/32 [==============================] - 1s 45ms/step - loss: 14.6215 - rmse: 3.8238
Epoch 74/100
32/32 [==============================] - 1s 45ms/step - loss: 16.7923 - rmse: 4.0978
Epoch 75/100
32/32 [==============================] - 1s 45ms/step - loss: 23.6171 - rmse: 4.8597
Epoch 76/100
32/32 [==============================] - 1s 44ms/step - loss: 20.1067 - rmse: 4.4840
Epoch 77/100
32/32 [==============================] - 1s 45ms/step - loss: 23.4237 - rmse: 4.8398
Epoch 78/100
32/32 [==============================] - 1s 44ms/step - loss: 25.5370 - rmse: 5.0534
Epoch 79/100
32/32 [==============================] - 1s 44ms/step - loss: 33.7498 - rmse: 5.8095
Epoch 80/100
32/32 [==============================] - 1s 44ms/step - loss: 22.2081 - rmse: 4.7125
Epoch 81/100
32/32 [==============================] - 1s 46ms/step - loss: 34.0196 - rmse: 5.8326
Epoch 82/100
```

```
32/32 [==============================] - 1s 45ms/step - loss: 25.7848 - rmse: 5.0779
Epoch 83/100
32/32 [==============================] - 1s 45ms/step - loss: 23.4041 - rmse: 4.8378
Epoch 84/100
32/32 [==============================] - 1s 45ms/step - loss: 23.9674 - rmse: 4.8956
Epoch 85/100
32/32 [==============================] - 1s 44ms/step - loss: 29.0349 - rmse: 5.3884
Epoch 86/100
32/32 [==============================] - 1s 44ms/step - loss: 21.8591 - rmse: 4.6754
Epoch 87/100
32/32 [==============================] - 1s 44ms/step - loss: 14.2806 - rmse: 3.7790
Epoch 88/100
32/32 [==============================] - 1s 44ms/step - loss: 18.8377 - rmse: 4.3402
Epoch 89/100
32/32 [==============================] - 1s 44ms/step - loss: 13.1473 - rmse: 3.6259
Epoch 90/100
32/32 [==============================] - 1s 44ms/step - loss: 9.5658 - rmse: 3.0929
Epoch 91/100
32/32 [==============================] - 1s 44ms/step - loss: 19.8885 - rmse: 4.4597
Epoch 92/100
32/32 [==============================] - 1s 45ms/step - loss: 14.8018 - rmse: 3.8473
Epoch 93/100
32/32 [==============================] - 1s 44ms/step - loss: 7.1501 - rmse: 2.6740
Epoch 94/100
32/32 [==============================] - 1s 44ms/step - loss: 18.9983 - rmse: 4.3587
Epoch 95/100
32/32 [==============================] - 1s 44ms/step - loss: 14.1897 - rmse: 3.7669
Epoch 96/100
32/32 [==============================] - 1s 44ms/step - loss: 11.6128 - rmse: 3.4078
Epoch 97/100
32/32 [==============================] - 1s 44ms/step - loss: 21.4982 - rmse: 4.6366
Epoch 98/100
32/32 [==============================] - 1s 44ms/step - loss: 23.1815 - rmse: 4.8147
Epoch 99/100
32/32 [==============================] - 1s 44ms/step - loss: 18.3960 - rmse: 4.2891
Epoch 100/100
32/32 [==============================] - 1s 44ms/step - loss: 10.4749 - rmse: 3.2365
```

You might receive the following error while fitting the neural network.

```
InvalidArgumentError: Unable to parse tensor proto
```

If you do receive this error, it is likely because you are missing proper authentication to access Google Drive to store your datasets.

In [ ]:

```

```